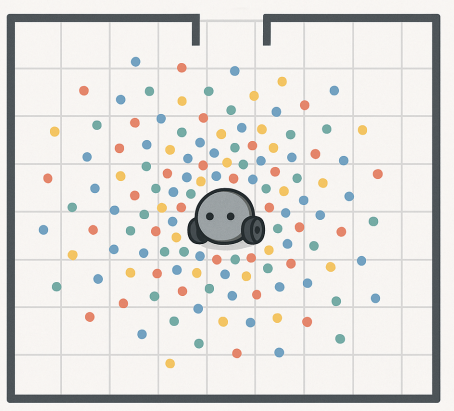
Zaawansowana Robotyka Inteligentna

Ćwiczenie 1

Lokalizacja robotów mobilnych za pomoca filtru cząsteczkowego (Particle filtering)



Zadanie do wykonania

Ćwiczenie składa się z trzech części:

1. Przeanalizuj na kartce krok po kroku toy example lokalizacji cząsteczkowej (z sekcji Toy Example filtru 1D). Następnie zaimplementuj filtr 1D na zadaniej w przykładzie mapie oraz zweryfikuj działanie na innej podobnej.

2. Zaimpelementuj analogiczny filtr 2D. Można korzystać z generatywnej AI. W sekcji (Przykładowy wariant dla zadania 2) znajdują się przykładowe scenariusze wykoanania zadania ale nie jest konieczne ich stosowanie. Konieczne jest wytłumaczenie działania kodu.

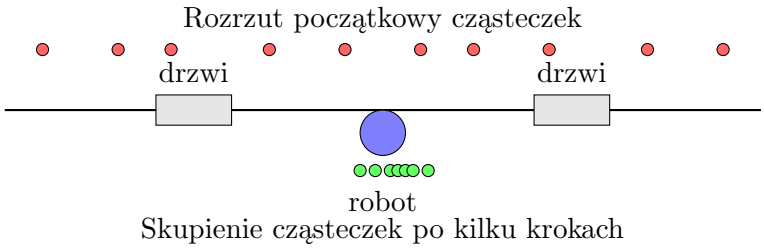
3. Część Black Box – stwórz aplikację demonstrującą filtr cząsteczkowy 2D lub 3D opatrzoną stosowną grafiką. W sekcji (Przykładowy scenariusz do zadania 3) znajduje się przykłądowy wariant wykonania zadania ale nie jest konieczne jego stosowanie. Wytłumacz jak działa demonstracja, zrozumienie kodu nie jest konieczne.

Toy Example filtru 1D

Rozważmy cykliczny świat o 5 polach:

$$\text{świat} = [\,\text{ściana},\ \text{drzwi},\ \text{ściana},\ \text{ściana},\ \text{drzwi}\,]$$

Robot porusza się w prawo (aktualna_pozycja $+1 \pmod 5$). Sensor rozróżnia *drzwi* i *ścianę* z dokładnością 0.8.



Rysunek 1: Idea filtru cząsteczkowego: początkowo hipotezy/cząsteczki (czerwone) są rozproszone, po kilku krokach koncentrują się wokół rzeczywistej pozycji robota (zielone).

Krok 1 – Inicjalizacja

Generujemy 10 cząsteczek w sposób równomierny:

$$[0, 1, 2, 3, 4, 0, 2, 3, 4, 1]$$

(liczby oznaczają indeksy pól w świecie).

Krok 2 – Pierwszy pomiar: drzwi

Wagi:

$$w_i = \begin{cases} 0.8 & \text{jeśli pole to drzwi (1 lub 4)} \\ 0.2 & \text{w przeciwnym razie} \end{cases}$$

Wagi cząsteczek przed normalizacją:

$$w = [0.2,\ 0.8,\ 0.2,\ 0.2,\ 0.8,\ 0.2,\ 0.2,\ 0.2,\ 0.8,\ 0.8],$$

$$S = \sum_{i=0}^{9} w_i = 4.4.$$

Po normalizacji:

$$w_i' = \frac{w_i}{S}$$

$$[\frac{0.2}{4.4}, \frac{0.8}{4.4}, \frac{0.2}{4.4}, \frac{0.2}{4.4}, \frac{0.8}{4.4}, \frac{0.2}{4.4}, \frac{0.2}{4.4}, \frac{0.2}{4.4}, \frac{0.8}{4.4}, \frac{0.8}{4.4}]$$

$$[0.045,\ 0.182,\ 0.045,\ 0.045,\ 0.182,\ 0.045,\ 0.045,\ 0.045,\ 0.182,\ 0.182].$$

Sprawdzenie sumy:

$$4 \times 0.182 + 6 \times 0.045 = 0.728 + 0.270 = 0.998 \approx 1.0.$$

Resampling (ruletka, z podaniem u i CDF)

Weźmy losowe wartości:

$$u = [0.05,\ 0.18,\ 0.32,\ 0.41,\ 0.55,\ 0.62,\ 0.71,\ 0.86,\ 0.93,\ 0.99].$$

Tworzymy z wag cząsteczek postać CDF (Cumulative Distribution Function), dystrybuantę — funkcję opisującą, z jakim prawdopodobieństwem zmienna losowa X przyjmie wartość nie większą niż dany próg x. W praktyce definiujemy kumulatywną sumę wag cząsteczek, która tworzy przedziały stosowane do losowania. Czym większa waga cząsteczek tym większy przedział w CDF, czyli z większym prawdopodobieństwem może zostać powielona podczas losowania. Nasze znormalizowane wagi cząsteczek, tworzą następującą listę CDF.

$$\text{CDF} \approx [0.045, 0.227, 0.272, 0.317, 0.499, 0.544, 0.589, 0.634, 0.816, 0.998],$$

dla każdego u_j wybieramy najmniejszy indeks i, dla którego $\text{CDF}_i \geqslant u_j$.

Tabela 1: Przyporządkowanie wartości u_j do indeksów CDF w metodzie ruletkowej.

u_j	Przedział CDF	Wybrany indeks i
0.05	$0.045 < u \leqslant 0.227$	1
0.18	$0.045 < u \leqslant 0.227$	1
0.32	$0.317 < u \leqslant 0.499$	4
0.41	$0.317 < u \leqslant 0.499$	4
0.55	$0.544 < u \leqslant 0.589$	6
0.62	$0.589 < u \leqslant 0.634$	7
0.71	$0.634 < u \leqslant 0.816$	8
0.86	$0.816 < u \leqslant 0.998$	9
0.93	$0.816 < u \leqslant 0.998$	9
0.99	$0.816 < u \leqslant 0.998$	9

Otrzymujemy wektor indeksów cząsteczek:

$$[1,\ 1,\ 4,\ 4,\ 6,\ 7,\ 8,\ 9,\ 9,\ 9].$$

Mapowanie indeksów na pozycje w świecie. Korzystamy z listy inicjalizacyjnej

$$[0,\ 1,\ 2,\ 3,\ 4,\ 0,\ 2,\ 3,\ 4,\ 1],$$

co daje:

$$1 \mapsto 1,\quad 1 \mapsto 1,\quad 4 \mapsto 4,\quad 4 \mapsto 4,\quad 6 \mapsto 2,\quad 7 \mapsto 3,\quad 8 \mapsto 4,\quad 9 \mapsto 1,\quad 9 \mapsto 1,\quad 9 \mapsto 1.$$

Nowe cząsteczki (pozycje w świecie):

$$[1,\ 1,\ 4,\ 4,\ 2,\ 3,\ 4,\ 1,\ 1,\ 1].$$

Krok 3 – Pierwszy ruch: +1 w prawo (ruch deterministyczny)

Aktualne cząsteczki (po resamplingu):

$$[1,\ 1,\ 4,\ 4,\ 2,\ 3,\ 4,\ 1,\ 1,\ 1].$$

Po wykonaniu ruchu o +1 w cyklicznym świecie ($+1 \pmod 5$):

$$[1,\ 1,\ 4,\ 4,\ 2,\ 3,\ 4,\ 1,\ 1,\ 1] \xrightarrow{+1} [2,\ 2,\ 0,\ 0,\ 3,\ 4,\ 0,\ 2,\ 2,\ 2].$$

Rozkład po ruchu:

$$0{:}3, \quad 2{:}5, \quad 3{:}1, \quad 4{:}1 \quad \text{(pozostałe 0)}.$$

Krok 4 – Drugi pomiar: ściana

Ściana występuje na pozycjach

$$\{0,\ 2,\ 3\}.$$

Po kroku 3 nasze cząsteczki znajdują się w pozycjach:

$$[2,\ 2,\ 0,\ 0,\ 3,\ 4,\ 0,\ 2,\ 2,\ 2].$$

Dziewięć z nich $(2, 2, 0, 0, 3, 0, 2, 2, 2)$ znajduje się w pozycjach odpowiadających obserwacji (**ściana**), natomiast jedna (4) — w pozycji **drzwi**. Zatem, przy dokładności sensora $p(\text{poprawny odczyt}) = 0.8$:

$$w = [0.8, 0.8, 0.8, 0.8, 0.8, 0.2, 0.8, 0.8, 0.8, 0.8]$$

Suma wag:

$$S = 9 \times 0.8 + 1 \times 0.2 = 7.4.$$

po normalizacji

$$w' \approx [0.108,\ 0.108,\ 0.108,\ 0.108,\ 0.108,\ 0.027,\ 0.108,\ 0.108,\ 0.108,\ 0.108].$$

$$\text{CDF} \approx [0.108,\ 0.216,\ 0.324,\ 0.432,\ 0.540,\ 0.567,\ 0.675,\ 0.783,\ 0.891,\ 1.000]$$

Losując wartości,

$$u = [0.05,\ 0.12,\ 0.19,\ 0.24,\ 0.41,\ 0.43,\ 0.74,\ 0.88,\ 0.93,\ 0.97]$$

Stosując do przedziałów CDF wybieramy indexy: [0,1,1,2,3,3,7,8,8,9], po resamplingu mamy cząsteczki:

$$[2,\ 2,\ 2,\ 0,\ 0,\ 0,\ 2,\ 2,\ 2,\ 2].$$

Rozkład po resamplingu:

$$0{:}3, \quad 2{:}7, \quad \text{(pozostałe 0)}.$$

Krok 5 – Drugi ruch: +1 w prawo

Aktualne cząsteczki (po drugim pomiarze):

$$[2, 2, 2, 0, 0, 0, 2, 2, 2, 2].$$

Po wykonaniu ruchu o +1 w cyklicznym świecie (+1 (mod 5)):

$$[2, 2, 2, 0, 0, 0, 2, 2, 2, 2] \xrightarrow{+1} [3, 3, 3, 1, 1, 1, 3, 3, 3, 3].$$

Rozkład po ruchu:

$$1{:}3, \quad 3{:}7, \quad \text{(pozostałe 0)}.$$

Krok 6 – Trzeci pomiar: ściana

Oczekiwane pole: **ściana** występuje w pozycjach

$$\{0, 2, 3\}.$$

Po kroku 5 nasze cząsteczki znajdują się na pozycjach:

$$[3, 3, 3, 1, 1, 1, 3, 3, 3, 3].$$

Z dokładnością sensora $p(\text{poprawny odczyt}) = 0.8$ i $p(\text{błędny odczyt}) = 0.2$ otrzymujemy:

$$\mathrm{w} = [0.8, 0.8, 0.8, 0.2, 0.2, 0.2, 0.8, 0.8, 0.8, 0.8].$$

Suma wag:

$$S = 7{\cdot}0.8 + 3{\cdot}0.2 = 5.6 + 0.6 = 6.2.$$

Po normalizacji:

$w_i' = [0.12903, 0.12903, 0.12903, 0.03226, 0.03226, 0.03226, 0.12903, 0.12903, 0.12903, 0.12903]$.

Resampling z CDF.

CDF = [0.1290, 0.2581, 0.3871, 0.4194, 0.4516, 0.4839, 0.6129, 0.7419, 0.8710, 1.0000]

Dla wartości losowych

$$u = [0.05, 0.12, 0.20, 0.33, 0.45, 0.60, 0.72, 0.81, 0.93, 0.99],$$

przyporządkowania (minimalny indeks i taki, że $u \leqslant \mathrm{CDF}_i$) są:

$$\boxed{[0, 0, 1, 2, 4, 6, 7, 8, 9, 9]}.$$

Stosując aktualny zbiór cząsteczek [3, 3, 3, 1, 1, 1, 3, 3, 3, 3], finalne cząsteczki po resamplingu wyglądają następująco:

$$\boxed{[3, 3, 3, 3, 1, 3, 3, 3, 3, 3]}.$$

Masa rozkładu wyraźnie koncentruje się na polu **3** (*ściana*), co oznacza, że robot zlokalizował się z wysokim prawdopodobieństwem w tym miejscu. Czyli jego pozycja startowa to drzwi pod indeksem 1.

Przykładowy wariant dla zadania 2

Implementacja filtru 2D

Mapa i reprezentacja

Można rozważyć siatkę $M \times N$ (np. 20×20) z komórkami typu: `wall`, `free`, `door`. Robot porusza się w przestrzeni ciągłej *po* tej siatce, lecz estymację pozycji prowadzimy w układzie lokalizacji (x, y, θ) lub uproszczonym (x, y).

Polityka (model) ruchu

Proponowana polityka kontrolna na krok $t \to t{+}1$:

- Z prawdopodobieństwem 0.7 wykonaj ruch *na wprost* o odcinek d (np. $d = 1$ komórka mapy lub 0.5 komórki mapy).
- Z prawdopodobieństwem 0.2 skręć w prawo o 90° i idź o d.
- Z prawdopodobieństwem 0.1 skręć w lewo o 90° i idź o d.

Szum ruchu: dodaj odchylenie Gaussa $\mathcal{N}(0, \sigma_d^2)$ do przebytego dystansu (np. $\sigma_d = 0.1\, d$) oraz $\mathcal{N}(0, \sigma_\theta^2)$ do kąta (np. $\sigma_\theta = 5^\circ$).

Model sensora (dokładność odczytu)

Dwie typowe opcje:

1. **Czujnik kategorii komórki** (prosty): mierzy typ komórki przed robotem (`door`/`wall`/`free`). Prawdopodobieństwa: $P(\text{poprawnie}) = 0.85$, a pozostałe klasy dzielą 0.15.
2. **Czujnik odległości do landmarków** (realistyczny): w mapie dodaj punkty orientacyjne (landmarki). Pomiar to odległość r i (opcjonalnie) kąt φ do najbliższego landmarku, z zakłóceniami Gaussa: $r \sim \mathcal{N}(r^*, \sigma_r^2)$, $\varphi \sim \mathcal{N}(\varphi^*, \sigma_\varphi^2)$, np. $\sigma_r = 0.2$ komórki, $\sigma_\varphi = 3^\circ$.

Wagi cząsteczek wyznaczaj na podstawie wiarygodności pomiaru (likelihood), np. gęstości Gaussa względem przewidywanej wartości z danej cząsteczki.

Dodatkowe Wskazówki do implementacji 2D

- Reprezentuj cząsteczkę jako wektor (x, y, θ, w).
- Resampling: systematyczny lub niskodyspersyjny (*systematic/stratified resampling*) ogranicza degenerację.
- Zwizualizuj: mapa + chmura cząsteczek + trajektoria prawdziwego robota.

Przykładowy scenariusz do zadania 3

Przykłądowe wykoanie zadania np. w Pythonie:

1. Wczytaj mapę 2D lub 3D), generuje rozkład cząsteczek.
2. W pętli wykonuj: *sense* $\rightarrow$ *normalize* $\rightarrow$ *resample* $\rightarrow$ *move*.
3. Po każdym kroku wyświetla: chmurę (x, y) (2D) lub (x, y, z) (3D) oraz estymatę pozycji (np. średnia ważona).
4. Daj możliwość zmieny parametrów: liczby cząsteczek N, dokładność sensora, wariant resamplingu, poziom szumu ruchu.

Demo może pokazywać *zbieżność* rozkładu cząsteczek do prawdziwej trajektorii robota i wpływ parametrów (szumów, liczby cząsteczek) na jakość lokalizacji.